

When Caching Meets Routing:

A Cache-Aware Cost Model for Selective Frontier-Model Routing in Multi-Turn Voice Agents

Michael Kaminski

Modular Equity, Atlanta, GA

michael@modularequity.com

Technical white paper · September 28, 2026 · Preprint, not peer-reviewed

Abstract

Routing the easy turns in an LLM workload to a cheaper model and saving the frontier model for hard turns is widely reported to cut inference cost by 35–98% at near-frontier quality [1, 2, 4]. These estimates generally treat each query as independent and priced at list rates. Multi-turn conversational agents, such as contact-center voice agents, break both assumptions: every turn re-sends a long, mostly unchanged context, and providers bill the re-sent prefix at a steep discount through *prompt caching*. Caches, however, are kept per model. We build a turn-level simulator that combines published prices (retrieved 2026-09-28), prompt-cache rules, a Markov process for turn difficulty, and an imperfect router with escalation. With caching switched off, it reproduces a closed-form list-price model to within 1.1 percentage points. Under a representative baseline (12 turns, 6,000-token cached system prompt, 20% hard turns), per-turn routing from Claude Opus 5.5 to Haiku 4.5 saves 51.3% under the naive model but only 9.2% once caching is modeled. Routing Opus 5.5 to Sonnet 5.5 moves from 33.8% savings to a net cost *increase* of -15.5% . Across 600 Monte Carlo scenarios, median cache-aware savings are 6.5% (90% interval -3.4% to 18.3%), against a naive median of 46.7%. Routing loses money in 16.2% of scenarios. The cause is *cache fragmentation*: tokens appended while a model is idle must be re-written to that model’s cache at $1.25\times$ input price. Routing whole conversations instead of individual turns avoids the penalty and recovers up to 70.0% savings on conversations routed to the small model. Every number in this paper is regenerated by a public script, and no proprietary or employer data is used.

1 Introduction

What we mean by “frontier.” A *frontier model* is a general-purpose model at the current edge of capability. It is usually trained with the largest compute budgets and leads broad reasoning, coding and agentic benchmarks. Architecture (transformer, matrix-multiply-heavy) does not define the class, because nearly every modern neural network shares it. Frontier models are also typically the *slowest and most expensive per token*, not the fastest. Regulators have drawn the boundary by training compute; the EU AI Act, for example, presumes systemic risk above 10^{25} FLOPs [8]. This definition matters because the obvious cost strategy for production agents follows from it: use the frontier model only where its capability is needed.

The routing thesis. LLM routing and cascading try to do exactly that. FrugalGPT reports up to 98% cost reduction while matching GPT-4 [1]. RouteLLM reports cost reductions of over 85% on MT-Bench, 45% on MMLU and 35% on GSM8K while keeping 95% of GPT-4’s quality [2, 3]. Hybrid LLM reports up to 40% fewer calls to the large model with no drop in quality [4]. RouterBench supplies over 405k inference outcomes for evaluating routers [5]. These results are real, but they come from *single-turn* benchmarks priced at *list* rates.

The gap. A voice agent in collections or servicing is multi-turn by construction. Each turn re-sends a system prompt, tool schemas, policy text and the growing transcript. Providers discount a re-sent prefix heavily: Anthropic bills cache reads at $0.05\text{--}0.1\times$ base input price [7]. A cache hit requires an identical prefix, and the cached state is the attention keys and values computed by one model, so a cache written by one model cannot be read by another: caches are, in effect, *isolated per model*. We therefore ask:

How much of the routing savings reported in the literature survives in a multi-turn agent once prompt caching and per-model cache isolation are modeled?

Contributions.

1. A turn-level, cache-aware cost model for routed multi-turn agents (§3), checked against a closed-form list-price model with caching switched off.
2. Evidence that caching removes most per-turn routing savings, and that routing can *raise* cost when the cheaper model’s cache-read price is not lower (§5).
3. A named mechanism, *cache fragmentation*, and a design rule: route at the granularity of the cache (the conversation), not the turn (§6).

2 Background: pricing and caching

Table 1 lists the public prices used. Two facts drive the results. First, the list-price ratio between the frontier and small model is $4\times$ (Opus 5.5 to Haiku 4.5) or $2\times$ (Opus 5.5 to Sonnet 5.5), but the cache-read ratio is only $2\times$ and $1\times$, because Opus 5.5 reads its cache at a reduced $0.05\times$ multiplier. Second, a cache hit requires an identical prefix, in practice on the *same model*, lasts 5 minutes by default (refreshed on use), and costs $1.25\times$ input to write. Haiku 4.5 will not cache prompts shorter than 4,096 tokens [7].

Table 1: Anthropic list prices, USD per million tokens, retrieved 2026-09-28 [6, 7].

Model	Input	Output	Cache write (5 min)	Cache read	Min. cacheable tokens
Claude Opus 5.5 (frontier)	4.00	20.00	5.00	0.20	512
Claude Sonnet 5.5	2.00	10.00	2.50	0.20	512
Claude Haiku 4.5	1.00	5.00	1.25	0.10	4,096

3 Model

Conversation. A conversation has T turns. The prompt at turn t has length $L_t = S + \sum_{k < t} (u + o + \tau_k) + u$, where S is the static system-plus-tools prefix, u the caller utterance, o the agent response and τ_k the tool-result tokens.

Turn difficulty. Hard turns (those that need frontier reasoning) follow a two-state Markov chain with persistence $\phi = \Pr(h_t = 1 \mid h_{t-1} = 1)$ and stationary share π . This requires $\Pr(h_t = 1 \mid h_{t-1} = 0) = \pi(1 - \phi)/(1 - \pi)$.

Router. On every turn a small classifier call (200 input and 5 output tokens on Haiku 4.5) flags the turn as hard with probability TPR if it is hard and FPR if it is easy. Flagged turns go to the frontier model F and unflagged turns go to the small model s . A hard turn sent to s is caught by a self-check with probability e and re-answered by F , so both calls are billed. Otherwise it is *missed*, which is a quality loss.

Naive (list-price) model. If caching and router cost are ignored and $\rho = p_s^{\text{out}}/p_F^{\text{out}}$ (equal to $p_s^{\text{in}}/p_F^{\text{in}}$ for these pairs), expected cost relative to frontier-only is

$$\kappa = \pi [\text{TPR} + (1 - \text{TPR})(\rho + e)] + (1 - \pi) [\text{FPR} + (1 - \text{FPR})\rho], \quad \text{savings} = 1 - \kappa. \quad (1)$$

Cache-aware model. Let ℓ_m be the warm prefix for model m : the prompt length at m ’s previous call if that call was within the TTL, and 0 otherwise. One call to m costs

$$c_m(L, \ell_m) = \begin{cases} \ell_m r_m + (L - \ell_m) w_m + o p_m^{\text{out}}, & L \geq L_m^{\text{min}} \\ L p_m^{\text{in}} + o p_m^{\text{out}}, & \text{otherwise,} \end{cases} \quad (2)$$

where r_m and $w_m = 1.25 p_m^{\text{in}}$ are the cache read and write prices. We assume the whole prompt is cached on every call, which is standard for multi-turn agents.

Cache fragmentation. Under frontier-only operation, each turn writes only the $\approx u + o + \tau$ new tokens. Under routing, when F returns after k turns on s , it must write every token appended during those k turns, $\Delta_F = \sum_j (u + o + \tau_j)$, at w_F instead of r_F . The small model likewise pays to write the full prefix S once. We call the extra write cost the *fragmentation penalty*. Eq. (1) leaves it out.

4 Experimental setup

Table 2 lists the baseline and Monte Carlo ranges. The parameters are *assumptions* chosen to represent a contact-center voice agent. They are not measurements from any deployment. Each point estimate averages 4,000 simulated conversations (fixed seeds). The Monte Carlo study draws 600 parameter vectors uniformly from the ranges and simulates 60 conversations for each.

Table 2: Simulation parameters (assumptions).

Symbol	Meaning	Baseline	MC range
T	turns per conversation	12	6–20
S	static cached prefix (system, tools, policy)	6,000	4,500–12,000
u, o	caller / agent tokens per turn	60, 80	30–100, 40–150
$\bar{\tau}$	mean tool-result tokens per turn (on 50% of turns)	150	0–400
π, ϕ	hard-turn share, persistence	0.20, 0.5	0.10–0.40, 0.2–0.8
TPR, FPR	router true/false positive rate	0.90, 0.15	0.80–0.97, 0.05–0.30
e	self-check escalation rate for missed hard turns	0.6	0.3–0.8
—	seconds per turn; cache TTL	20; 300	fixed

5 Results

Validation. With caching switched off, the simulator reproduces Eq. (1) to within 1.1 percentage points for every $\pi \in [0.05, 0.6]$ and both model pairs (Fig. 1, grey vs. dashed). For example, it gives 50.8% simulated against 51.3% analytic at baseline. The remaining gap is the router’s own cost. This checks the routing and escalation arithmetic, not the cache-billing path, which rests on the published pricing rules in §2.

Caching alone. Prompt caching cuts frontier-only cost from \$0.3865 to \$0.0819 per conversation, a 78.8% reduction. This single, near-free change dwarfs what any router can add at baseline.

Table 3: Baseline cost per conversation and savings versus frontier-only. The negative value marks a cost increase.

Policy	No caching		Cache-aware	
	Cost	Savings	Cost	Savings
Frontier-only (Opus 5.5)	\$0.3865	—	\$0.0819	—
Per-turn route → Haiku 4.5	\$0.1901	50.8%	\$0.0744	9.2%
Per-turn route → Sonnet 5.5	\$0.2580	33.3%	\$0.0947	−15.5%
Sticky escalation → Haiku 4.5	—	—	—	−0.2%
Whole conversation on Haiku 4.5	—	—	\$0.0246	70.0%
Whole conversation on Sonnet 5.5	—	—	\$0.0492	39.9%

Routing under caching. Table 3 and Fig. 1 show the main result. Per-turn routing to Haiku 4.5 sends 30.9% of turns to the frontier model and misses 0.8% of turns. Its savings fall from 50.8% to 9.2% once caching is modeled. Routing to Sonnet 5.5 *raises* cost by **−15.5%**, because Sonnet 5.5 reads its cache at the same \$0.20/MTok as Opus 5.5, so fragmentation writes outweigh the output-price savings. Fig. 2a breaks the cost down: routing halves output spend, but cache-write spend *grows*.

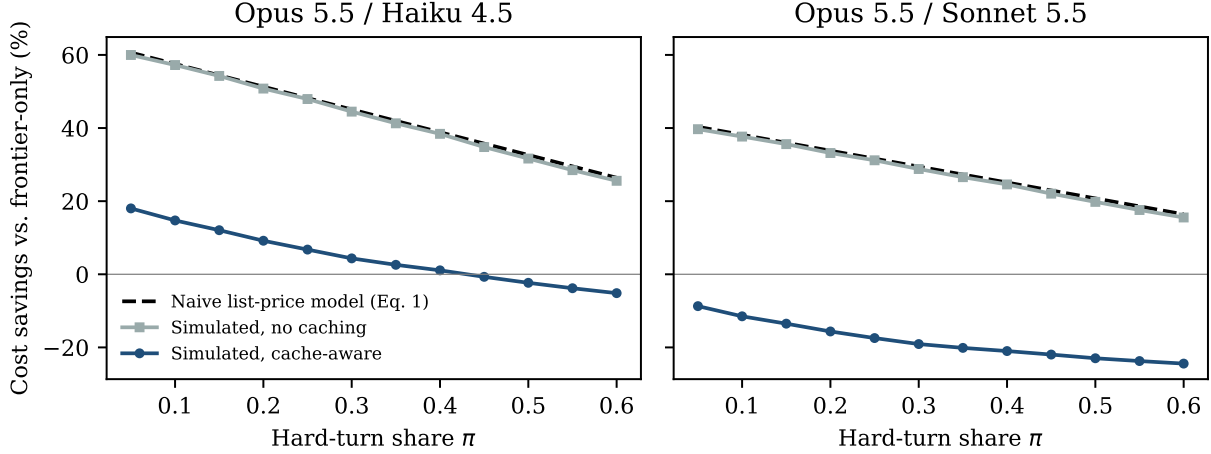


Figure 1: Savings versus hard-turn share π . Dashed: Eq. (1). Grey: simulator with caching off (validation). Blue: cache-aware simulator.

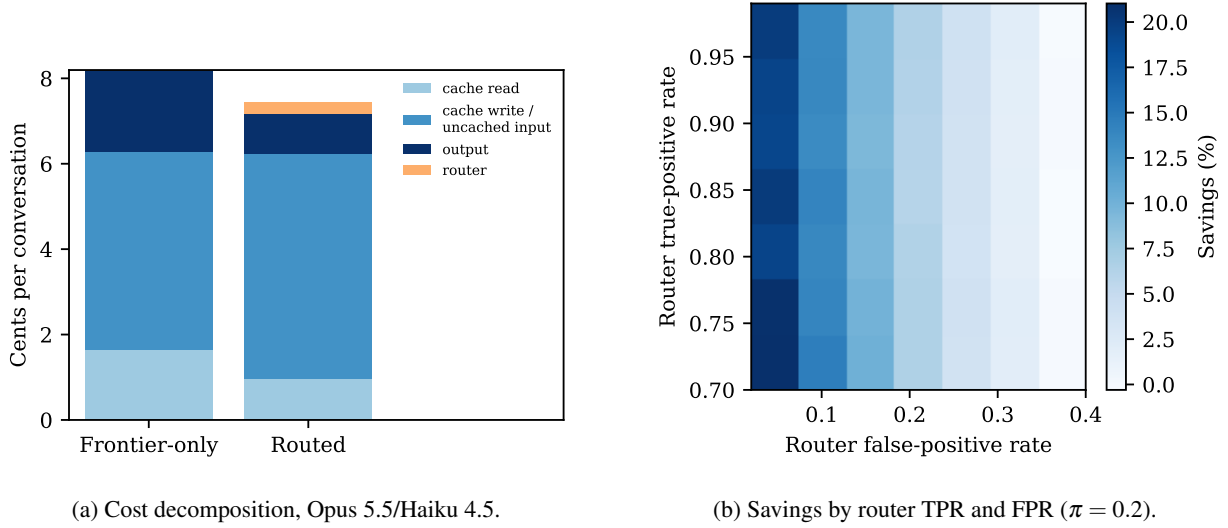


Figure 2: Where routing savings come from, and how much router quality matters.

Router quality. Across $\text{TPR} \in [0.70, 0.99]$ and $\text{FPR} \in [0.02, 0.40]$, cache-aware savings range from -0.3% to 21.0% (Fig. 2b). The false-positive rate matters far more than the true-positive rate. Every false positive pays full frontier price *and* a fragmentation write.

Uncertainty. Across 600 scenarios, cache-aware savings have a median of 6.5% and a 90% interval of -3.4% to 18.3% . The naive model’s median is 46.7% , so it overstates savings by a median of 40.2 percentage points (Fig. 3).

Routing increases cost in 16.2% of scenarios. The median missed-hard-turn rate is 1.0% (95th percentile 3.5%). Spearman correlations with savings are: FPR -0.52 , π -0.52 , agent output length o $+0.47$, turns T $+0.24$, prefix S -0.22 , TPR -0.02 . Routing pays best when output makes up a large share of cost and the cached prefix is small.

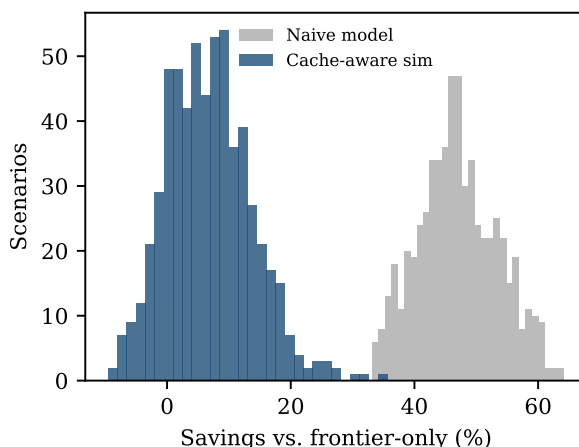


Figure 3: Distribution of savings across 600 parameter draws: naive model (grey) and cache-aware simulator (blue).

Alternative policies. *Sticky escalation* keeps a conversation on the frontier model once it has escalated, and stops calling the router. It avoids fragmentation but drifts 77.9% of turns to F , giving -0.2% (Haiku) and -10.4% (Sonnet). *Conversation-level routing* keeps each cache intact. A conversation served entirely by Haiku 4.5 costs 70.0% less than one on Opus 5.5. Realized savings then equal 70.0% times the share of conversations that can safely be assigned to the small model up front. At baseline, 18.4% of conversations contain no hard turn, which caps savings at about 12.9% even with a perfect conversation-level classifier.

Short prefixes. When S is below Haiku 4.5’s 4,096-token caching minimum, early small-model calls cannot be cached. Per-turn routing savings are -11.1% at $S=2,000$, 3.8% at 3,500, 11.0% at 4,500 and 7.5% at 8,000.

6 Discussion: design implications

1. **Cache first, route second.** Caching delivered a 78.8% reduction in this setting. Any routing business case should be measured against a *cached* frontier-only baseline, not list prices.
2. **Compare effective prices, not list prices.** The relevant ratio blends cache-read, cache-write and output prices, weighted by the workload’s token mix. A cheaper model with the same cache-read price (here Sonnet 5.5) can be a net loss.
3. **Route at cache granularity.** Assigning whole conversations, or long stable segments, avoids fragmentation. Per-turn routing should be limited to cases where the frontier call can be made *stateless*, with a short, cacheable, model-specific prompt.
4. **Optimize FPR.** A router tuned for recall pays twice for every false alarm.
5. **Keep deterministic work off LLMs.** Calculations, policy lookups and state updates cost nothing to route and belong in code.

7 Limitations and threats to validity

- **Assumed workload.** All token counts, difficulty rates and router accuracies are assumptions, not measurements. We encourage readers to replace them with their own traces.

- **One provider’s prices.** Other providers use different cache multipliers, TTLs and isolation rules, and prices change often. The mechanism generalizes, but the magnitudes may not.
- **Quality as a proxy.** Quality is measured only as a missed-hard-turn rate; we do not score response quality.
- **Speech costs excluded.** Speech-to-text, text-to-speech and telephony costs are omitted. They lower the share of LLM spend in total cost per call.
- **No invoice check.** The cache-aware cost path is derived from published billing rules and has not been reconciled against real API invoices.
- **Simplified caching.** We assume the whole prompt is cached on every call, with one breakpoint and a 5-minute TTL. More aggressive cache engineering could reduce the fragmentation penalty.

8 Conclusion

Published routing savings are measured on single-turn, list-priced benchmarks. Carried over to multi-turn agents, they overstate savings by a median of 40.2 percentage points in our scenarios, and can reverse sign. The cause is cache fragmentation across isolated per-model caches. The practical rule is to cache first, compare effective prices, and route at the granularity of the cache.

Reproducibility. `model.py` regenerates every figure and `results.json`; `macros.py` writes every number in the text into `numbers.tex`. The model uses Python 3 with NumPy, SciPy and Matplotlib.

Disclosure. The author leads a voice-AI initiative in consumer-finance servicing. This paper uses *no* employer data, systems or results. All inputs are public prices or stated assumptions.

References

- [1] L. Chen, M. Zaharia, J. Zou. FrugalGPT: How to use large language models while reducing cost and improving performance. arXiv:2305.05176, 2023.
- [2] I. Ong, A. Almahairi, V. Wu, W.-L. Chiang, T. Wu, J. E. Gonzalez, M. W. Kados, I. Stoica. RouteLLM: Learning to route LLMs with preference data. arXiv:2406.18665, 2024.
- [3] LMSYS Org. RouteLLM: An open-source framework for cost-effective LLM routing. <https://www.lmsys.org/blog/2024-07-01-routellm/>, July 2024.
- [4] D. Ding, A. Mallick, C. Wang, R. Sim, S. Mukherjee, V. Rühle, L. V. S. Lakshmanan, A. H. Awadallah. Hybrid LLM: Cost-efficient and quality-aware query routing. ICLR 2024. arXiv:2404.14618.
- [5] Q. J. Hu, J. Bieker, X. Li, N. Jiang, B. Keigwin, G. Ranganath, K. Keutzer, S. K. Upadhyay. RouterBench: A benchmark for multi-LLM routing system. arXiv:2403.12031, 2024.
- [6] Anthropic. Pricing. <https://platform.claude.com/docs/en/about-claude/pricing>, accessed 2026-09-28.
- [7] Anthropic. Prompt caching. <https://platform.claude.com/docs/en/build-with-claude/prompt-caching>, accessed 2026-09-28.
- [8] Regulation (EU) 2024/1689 (Artificial Intelligence Act), Art. 51. Official Journal of the European Union, 2024.